

IBAIT24

ATdIT

Technical Documentation

Group 4

Experience Management Dashboard

Alessandro DeSimone, Madelaine Martin,
Yoana Parvanova, Julius Schwab,
Charlotte Walter

Version 1

May 2026

Table of Contents

1.	PROJECT INFORMATION.....	1
1.1	STAKEHOLDERS AND ROLES.....	1
2.	DESIGN	1
2.1	KEY REQUIREMENTS	1
2.2	CONTEXT	2
2.3	MAJOR BUILDING BLOCKS.....	2
2.3.1	<i>Suggestion Resolution System.....</i>	<i>3</i>
2.3.2	<i>Customer Value (CV) Score Calculation.....</i>	<i>6</i>
2.3.3	<i>Incident Management and Priority Scoring</i>	<i>8</i>
2.3.4	<i>Simulation Engine.....</i>	<i>9</i>
2.4	INTERFACES / COMMUNICATION HANDLING	9
2.5	DESIGN CHALLENGES.....	10
2.6	USER INTERFACE / UX DESIGN.....	10
2.7	USED COMPONENTS AND FRAMEWORKS.....	11
2.8	PACKAGE / DEVELOPMENT COMPONENT CONCEPT	11
2.9	COMPLIANCE TO STANDARDS AND GUIDELINES	12
2.9.1	<i>Applied Architecture and Design Guidelines.....</i>	<i>12</i>
2.9.2	<i>Deviations.....</i>	<i>14</i>
3.	DESIGN DETAILS	14
3.1	BUSINESS OBJECTS / DATABASE DESIGN.....	14
3.2	TESTABILITY AND TEST ENVIRONMENT	16
3.3	COMPLEX ALGORITHMS AND APPLIED PATTERNS	16
3.4	DESIGN ALTERNATIVES AND TRADE OFFS.....	17
3.5	GUIDE TO THE IMPLEMENTATION.....	18
4.	APPENDIX.....	18
4.1	GLOSSARY	18
4.2	CUSTOMIZING	19
4.3	SUPPORTABILITY CONSIDERATIONS.....	19
4.4	ERROR ANALYSIS.....	19
4.4.1	<i>Debugging.....</i>	<i>19</i>
4.4.2	<i>Logging and Tracing</i>	<i>19</i>
4.4.3	<i>Other Error Analysis Tools.....</i>	<i>19</i>
4.5	OTHER.....	19
4.5.1	<i>Use of AI.....</i>	<i>19</i>

1. Project Information

1.1 Stakeholders and Roles

ROLE	NAME
AUTHOR(S)	Madelaine Martin
ARCHITECT(S)	Madelaine Martin Julius Schwab
UI	Madelaine Martin
MOCKUPS	Yoana Parvanova
PROCESS DESIGN	Charlotte Walter
ENDUSER-DOCUMENTATION	Alessandro DeSimone
PRESENTATION	Charlotte Walter Alessandro DeSimone

2. Design

Managing customer experience after service failures is essential for any business that relies on repeat customers and referrals. When incidents occur, whether it is a flight delay, a negative feedback score or an onboarding issue, the company needs a structured way to detect these problems, prioritize them based on customer value, and suggest appropriate recovery actions to advisors.

This system provides an advisor dashboard that monitors customer journeys in real time, detects incidents, calculates how important each customer is and recommends tier-specific recovery actions.

2.1 Key Requirements

The Experience Management system addresses the following core requirements:

- Track customers through their entire journey (from initial interest through post-flight feedback)
- Detect and manage service incidents such as flight delays, negative feedback, and onboarding issues
- Calculate a Customer Value (CV) score to prioritize high-value customers for recovery actions
- Suggest appropriate recovery actions based on the customer's booking tier (VIP, Gold, Standard)
- Monitor the company-wide Net Promoter Score (NPS) as the primary success metric
- Provide advisors with a real-time dashboard for managing assigned customers and incidents
- Simulate customer journeys for testing and demonstration purposes

2.2 Context

This software is part of a fictional space tourism company operated by “Elon Bezos”. The company suffers from low brand reputation, low customer loyalty and few referrals. The Experience Management module is responsible for retaining customer and improving the brand after service failures occur.

The system sits between all aspects of the customer lifecycle, after booking but between preparation, the actual flight and the feedback analysis. It handles what happens when things go wrong and ensures that the customers are recovered through proactive advisor interventions.

The critical KPI for this module is the Net Promoter Score (NPS), which measures how likely customers are to recommend the service. The NPS is calculated from the `referral_score` field of customer feedback entries: customers with scores of 9-10 are promoters, those with 0-6 are critics, and $NPS = \% \text{ promoters} - \% \text{ critics}$.

2.3 Major Building Blocks

The application follows a layered architecture with one-directional dependencies. Each layer depends on the layer below it, which ensures that changes in one layer do not propagate upward.

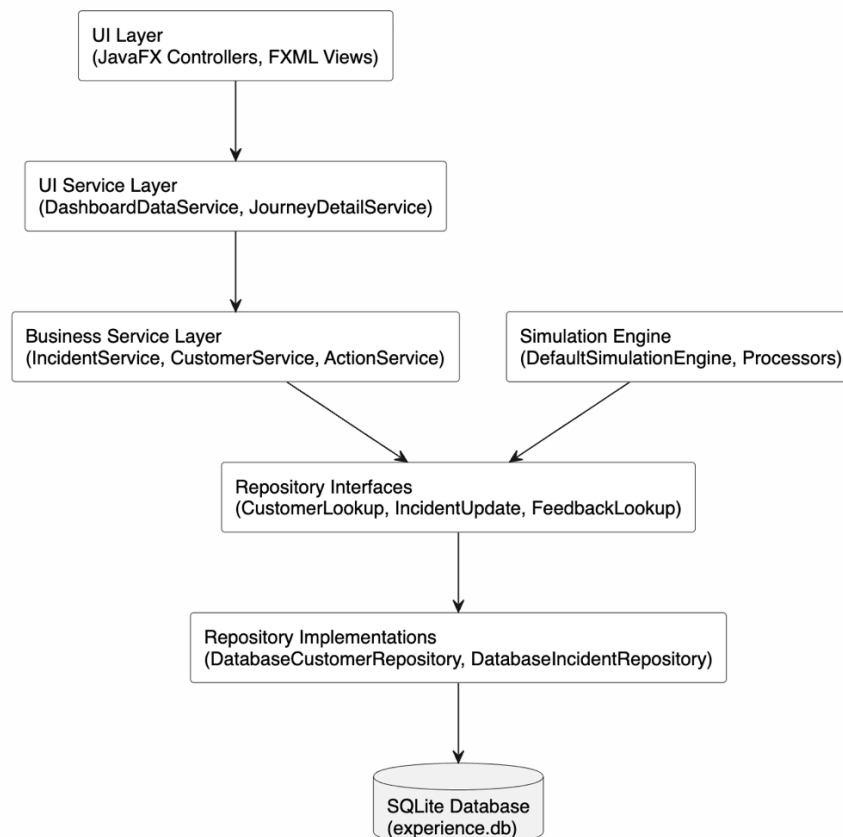


Figure 1: Layered architecture showing dependency direction

The identified layers from top to bottom are:

UI Layer: JavaFX controllers and FXML views handle user interaction, rendering, and navigation. The controllers are thin, they delegate all business logic to the service layer and only handle event binding and display updates.

UI Service Layer: This layer bridges the UI controllers with the business services. Classes like `DashboardDataService`, `JourneyDetailService`, and `FlightDetailService` aggregate data from multiple business services and provide it in a UI-friendly format.

Business Service Layer: Contains the core business logic. `IncidentService` manages incident lifecycle, `CustomerService` handles customer data, `ActionService` manages recovery actions, `NPSService` calculates the Net Promoter Score and `RecommendationExecutionService` handles the full flow of sending a recovery recommendation.

Repository Interfaces: Abstractions for data access, deliberately split by responsibility. For example, customer data access is split into `CustomerLookup` (read-only queries), `CustomerUpdate` (write operations), and `CustomerSearch` (filtered queries). This follows the Interface Segregation Principle.

Repository Implementation: Concrete SQLite database access using JDBC. Each implementation class translates method calls into SQL statements.

Database: SQLite file-based database that auto-created on startup.

Additionally, a Simulation Engine module runs in parallel to generate test data and simulate customer journeys for demo purposes.

2.3.1 Suggestion Resolution System

The suggestion system is responsible for determining which recovery actions to recommend when an incident is created. This is the core value proposition of the application, it takes a raw incident and produces concrete, actionable suggestions tailored to the customer's importance.

The design pattern behind this is a combination of the Strategy pattern with a factory approach. The interface `SuggestionResolver` defines two methods: `supports(Incident)` which checks whether a resolver can handle a given incident type, and `resolve(Incident)` which returns the appropriate `SuggestionStrategy`.

Three resolvers exist in the system:

- **DelaySuggestionResolver** handles delay incidents. It distinguishes between short delays (under 60 minutes) which receive a generic operational response via `DelayShortOperationalSuggestionStrategy` and longer delays which receive tier-specific suggestions determined by the customer's booking package.
- **FeedbackSuggestionResolver** handles negative feedback incidents. It uses a map of `FeedbackCategory` to `PackageSuggestionFactory` to select the correct factory based on the feedback category (FOOD, HOTEL, or FLIGHT), and then the factory selects the tier-specific strategy.
- **OnboardingSuggestionResolver** handles onboarding issues and always returns the same `OnboardingSuggestionStrategy` regardless of tier.

The `PackageResolver` interface (implemented by `FlightPackageResolver`) determines the customer's current booking package by looking up their current flight. This keeps the resolvers decoupled from the database layer.

The `PackageSuggestionFactory` holds a map of Packages (VIP, GOLD, STANDARD) to concrete `SuggestionStrategy` implementations. When called with a package tier, it returns the corresponding strategy.

Each `SuggestionStrategy` implementation is a simple class that sets two suggestion strings on an `ActionItem`. For example, `FoodVipSuggestionStrategy` sets:

- Suggestion 1: "Private Michelin Home-Cooking before the next flight"
- Suggestion 2: "Pre-Paid Table Reservation at a high class restaurant the day before the next flight"

The class `SuggestionService` orchestrates this entire flow. It receives a list of `SuggestionResolver` instances and iterates through them until one reports that it supports the given incident. It then calls `resolve()` to get the strategy and `apply()` to populate the action item. The following code shows this:

```
public void applySuggestions(ActionItem actionItem, Incident incident) {
    resolvers.stream()
        .filter(r -> r.supports(incident))
        .findFirst()
        .map(r -> r.resolve(incident))
        .orElseThrow(() -> new IllegalArgumentException(
            "No resolver found for incident type: " + incident.getType()))
        .apply(actionItem, incident);
}
```

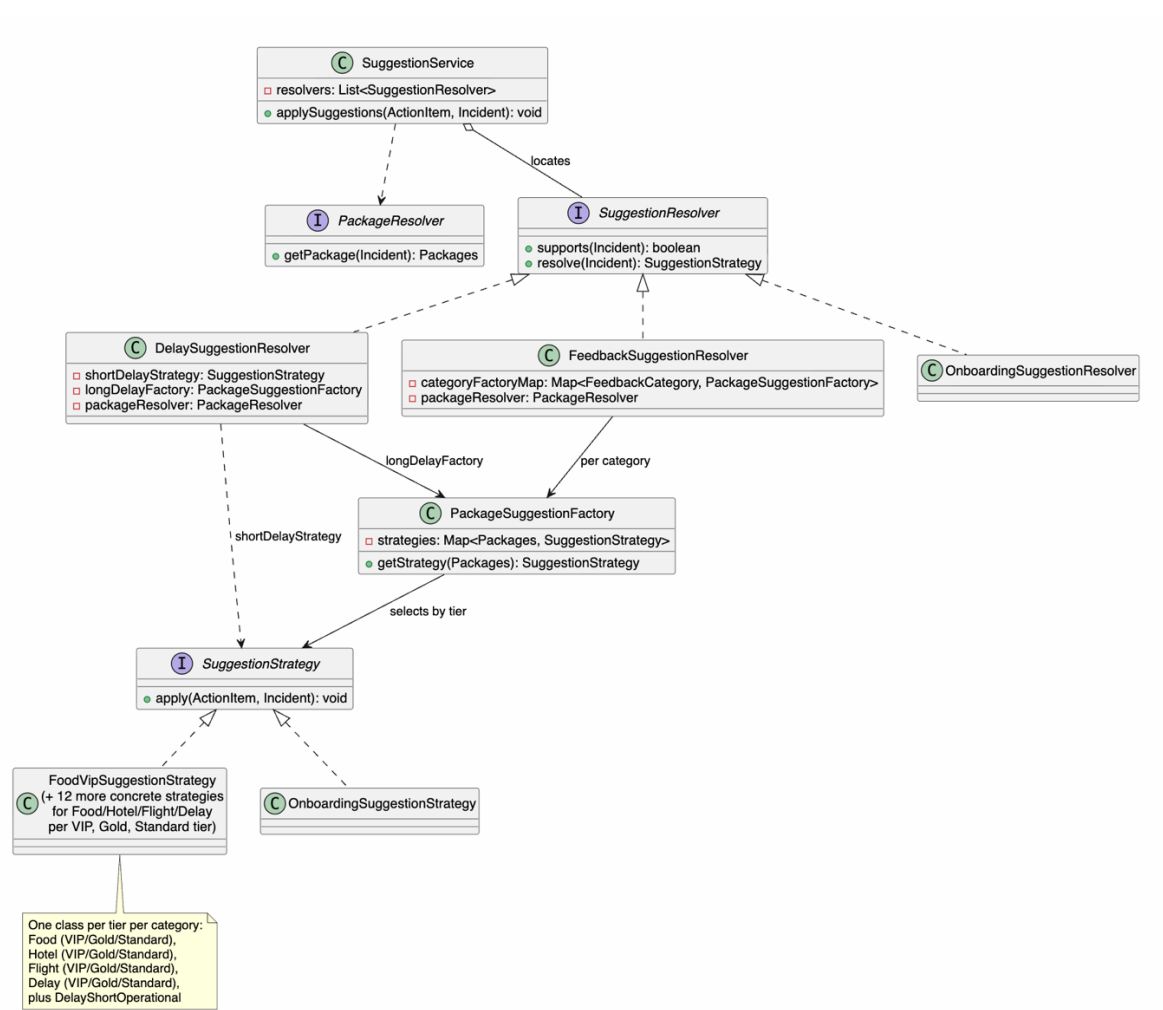


Figure 2: Class diagram showing the Suggestion Resolution system with Strategy, Factory, and Composite patterns

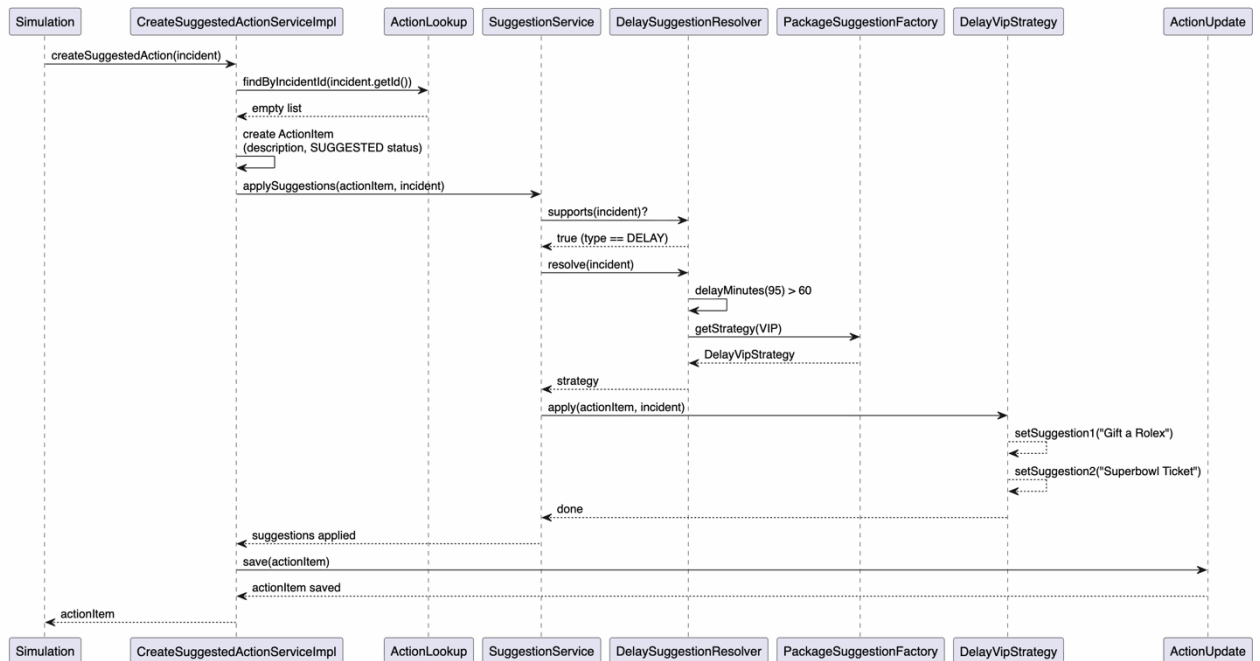


Figure 3: Sequence diagram showing how an incident triggers suggestion generation through the resolver chain

2.3.2 Customer Value (CV) Score Calculation

The CV score determines how important a customer is and directly influences incident prioritization. A higher CV score means the customer's incidents will appear higher in the advisor's queue.

The calculation is done by CVScoreCalcServiceImpl which receives a list of CVScoringRuleService implementations via constructor injection. Each rule is an independent scoring component that evaluates one aspect of the customer's profile and returns a point value.

The interface CVScoringRuleService defines a single method:

```

public interface CVScoringRuleService {
    int evaluate(CVCustomer cvc);
}
  
```

The CVCustomer class acts as a read-only aggregate that combines data from the Customer, CustomerCvProfile, and Flight entities into a single object suitable for rule evaluation. It holds a List of Flight objects where the first entry is the current flight and subsequent entries are previous flights. This is extensible for future scenarios where more than one previous flight may be relevant.

The following rules are configured:

Rule	Points	Condition
ReturningCustomerRule	+10	Customer has had a previous flight with the company
CurrentBookingStatusRule	+15	Current booking package is either Gold or VIP
LastBookingTierRule	+20	Previous booking was Gold or VIP
MarketingConsentRule	+15	Marketing consent given
BookingConnectionRule	+10	Referral code used
PaymentMethodRule	+10/+5/+0	IMMEDIATE / MONTHS / LONGTIME
PublicFigureRule	+15	Customer is a public person
TravelingAsScientistRule	+10	Customer type is Scientist
NewsletterSubscriptionRule	+5	Subscribed to the newsletter

The CVScoreCalcServiceImpl simply iterates through all rules and sums their results:

```
public int calculate(CVCustomer customer) {  
    int score = 0;  
    for (CVScoringRuleService rule : rules) {  
        score += rule.evaluate(customer);  
    }  
    return score;  
}
```

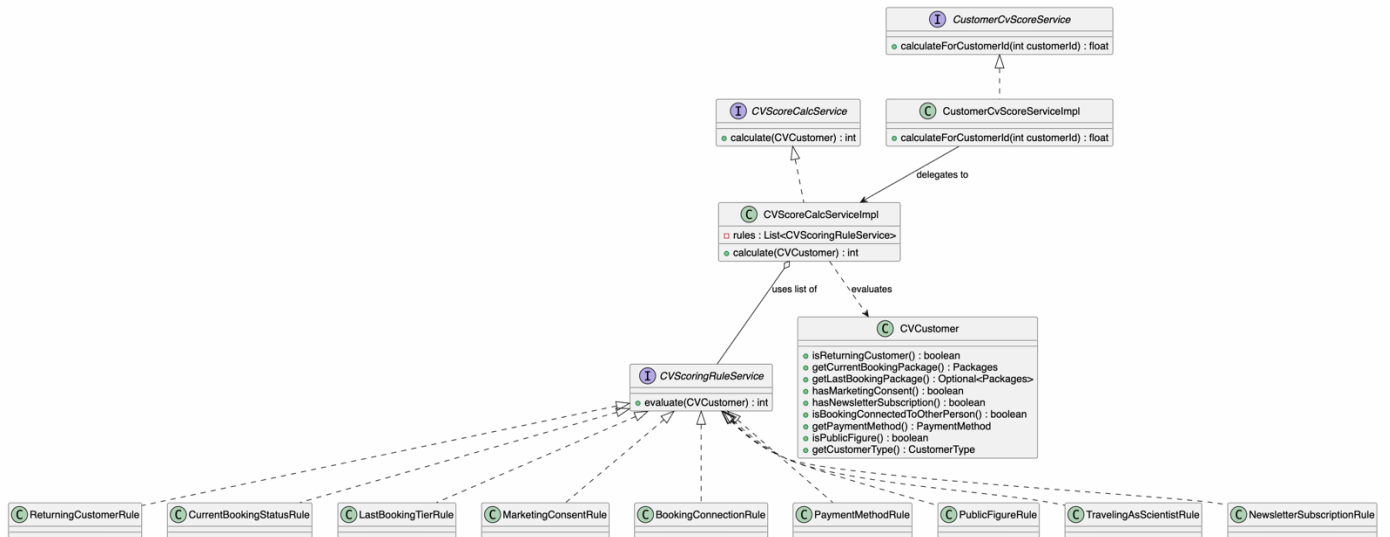


Figure 4: Class diagram showing CV Score calculation with injected rules following the Strategy pattern

This design is open for extension — new rules can be added by creating a new class implementing CVScoringRuleService and adding it to the list in BackendFactory. No existing code needs to change.

2.3.3 Incident Management and Priority Scoring

When the simulation detects a service failure, it creates an incident via one of the specialized creation services: CreateDelayIncidentServiceImpl, CreateFeedbackIncidentServiceImpl, or CreateOnboardingIncidentServiceImpl.

After an incident is created, CreateSuggestedActionServiceImpl is called to generate the recovery action with suggestions. The action item is saved and linked to the incident.

Incidents are prioritized using a priority score calculated by PriorityCalcServiceImpl. The score is based on:

- A base severity weight that varies by incident type (delays at 1.3, others at 1.5)
- The customer's CV score (higher-value customers get higher priority)
- The expected score impact and revenue risk

This ensures VIP customers with high-impact delays are addressed before standard customers with minor issues.

The priority score is calculated using the following formula:

$$\text{Priority Score} = (\text{BaseSeverity} \times \text{CVScore}) + (\text{ScoreImpact} \times 10) + (\text{RevenueImpact} / 2000)$$

Where BaseSeverity is 1.3 for DELAY incidents and 1.5 for FEEDBACK and ONBOARDING incidents. This ensures that customers with higher CV scores receive proportionally higher priority, while the flat ScoreImpact and RevenueImpact terms reflect the absolute severity of the incident type

2.3.4 Simulation Engine

The simulation engine generates realistic test data by simulating customer journeys. It consists of several processors:

- CustomerCreationProcessor creates new customers at configurable intervals with CV profiles, flights, and historical feedback for returning customers. Upon creation a customer gets assigned to the advisor with the least amount of customers.
- JourneyAdvancementProcessor advances customers through the status lifecycle (INTERESTED to COMPLETED).
- DelayIncidentGenerator probabilistically creates delay incidents at the PRE_FLIGHT stage.
- SimulationFeedbackGenerator generates feedback when customers reach the FEEDBACK stage.
- OnboardingIncidentGenerator creates incidents when customers are stuck in ONBOARDING beyond a threshold.

A key policy is OpenIncidentBlockPolicy: if a customer has an open incident, their journey does not advance until resolved.

The simulation engine operates on a tick-based model. Each time the simulation advances, the DefaultSimulationEngine calls tick(), which triggers the CustomerCreationProcessor and JourneyAdvancementProcessor sequentially. The following sequence diagram shows how a single tick leads to a delay incident being created and a suggested action being generated for it.

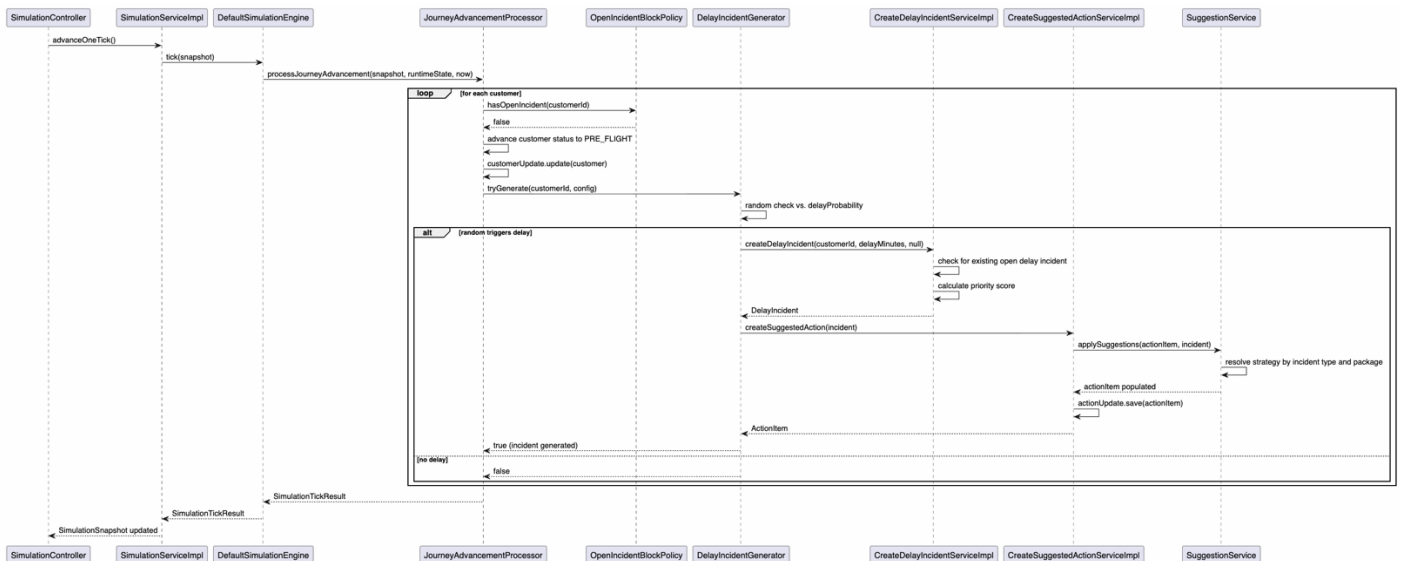


Figure 5: Simulation Tick - Delay Incident Generation

2.4 Interfaces / Communication Handling

All communication within the system is synchronous and in-process (single JVM, desktop application):

- **UI to Service:** Controllers call UI service methods directly.
- **Service to Repository:** Business services call repository interfaces; implementations handle SQL
- **Database:** SQLite accessed via JDBC through the ConnectionProvider abstraction.

- **Simulation to Repository:** Uses the same interfaces for consistent data access.

2.5 Design Challenges

The first challenge was dashboard auto-refresh conflicting with user input. The dashboard refreshes every second but this caused text inputs to lose content mid-typing. Solved by implementing `isUserTyping()` in `DashboardController` that skips detail pane refresh when a `TextInputControl` has focus.

The second challenge was suggestion system flexibility. Different tiers and incident types require different responses. Solved with the multi-level Strategy pattern (resolvers, factories, strategies) described in 2.3.1.

The third challenge was simulation data lifecycle. Temporary simulation data must be cleaned without affecting pre-seeded demo data. Handled by `DatabaseSimulationDataCleanupService`. Simulation-generated customers are removed by email pattern with related rows removed through foreign keys.

The fourth challenge was incident-based journey blocking. Open incidents must prevent journey advancement. Solved with `OpenIncidentBlockPolicy` checking the incident repository before each step.

2.6 User Interface / UX Design

The application has two main screens:

1. Simulation Control Screen: Configure simulation parameters and select an advisor to log in as.
2. Advisor Dashboard: Split into three panels
 - Left panel: list of open incidents or assigned customers
 - Center panel: detail view (incident details, customer journey, flight information)
 - Right panel: action panel (suggested recovery actions, recommendation emails)

Simulation Control
Space Flight Experience Management

Start Stop Clear Simulation Data Open Dashboard

Simulation Setup

Advisor: Anna Schmidt | Premium Support

Customer creation interval (s): 10

Journey advancement interval (s): 15

Pre-flight delay probability: 0.25

Minimum delay minutes: 30

Maximum delay minutes: 120

Low-score feedback probability: 0.30

Low-score threshold: 3

Onboarding stuck threshold (s): 60

Onboarding incident probability: 0.25

Simulation State

Status: **STOPPED**

Started: n/a

Stopped: n/a

Last update: n/a

Simulation Metrics

Created customers: 0

Advanced journeys: 0

Delay incidents: 0

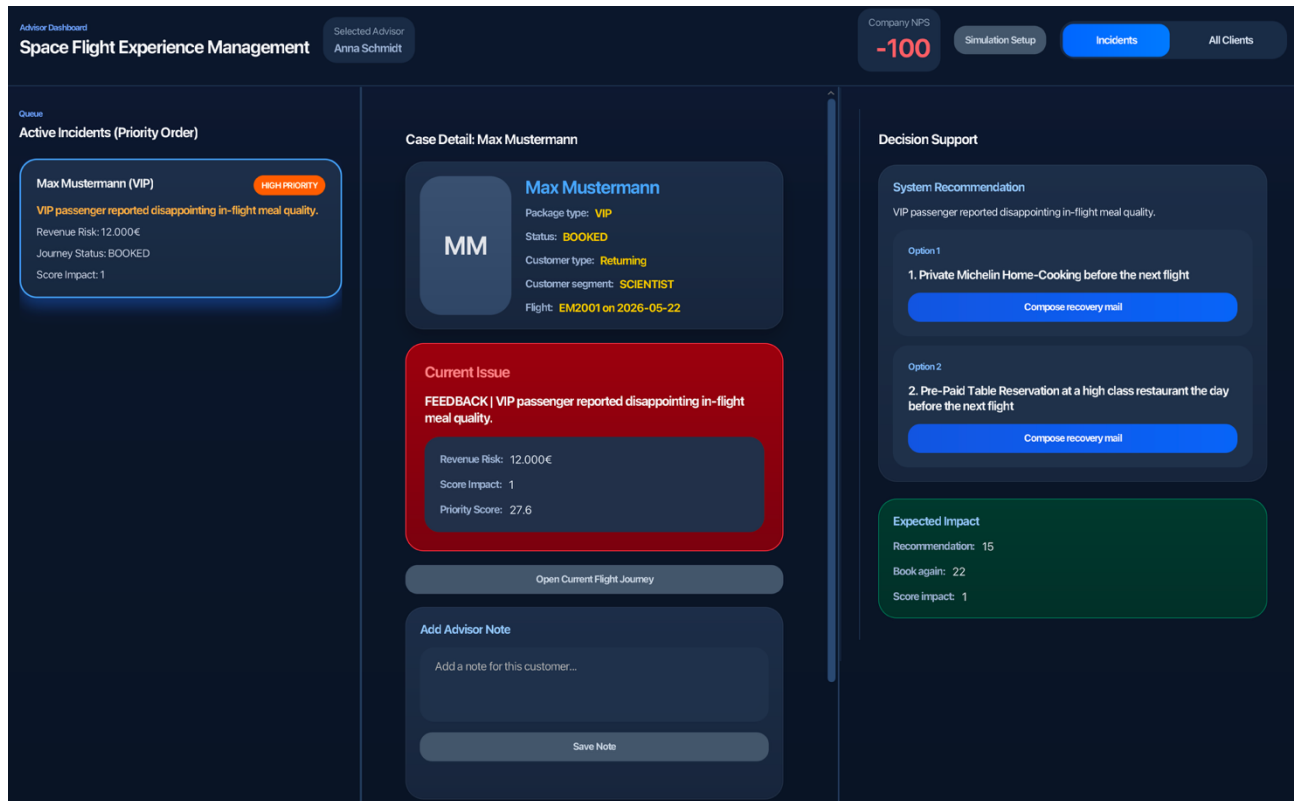
Onboarding incidents: 0

Generated feedbacks: 0

Feedback incidents: 0

Configure a simulation run or open the dashboard.

Screenshot of the Simulation Screen



Screenshot of the Dashboard – further explained in the End-User-Documentation

The dashboard supports two modes:

- Incidents Mode: Shows open incidents sorted by priority score
- Customers Mode: Shows assigned customers with their journey status. If a customer has an open incident that can still be seen in this mode including the suggestions on the right panel.

2.7 Used Components and Frameworks

Component	Purpose
JavaFX	Desktop UI Framework
SQLite (JDBC)	File-based database
SLF4J	Logging Framework
Maven	Build tool and dependency management

No additional third-party libraries are used beyond these approved components

2.8 Package / Development Component Concept

The project follows a domain-driven package structure:

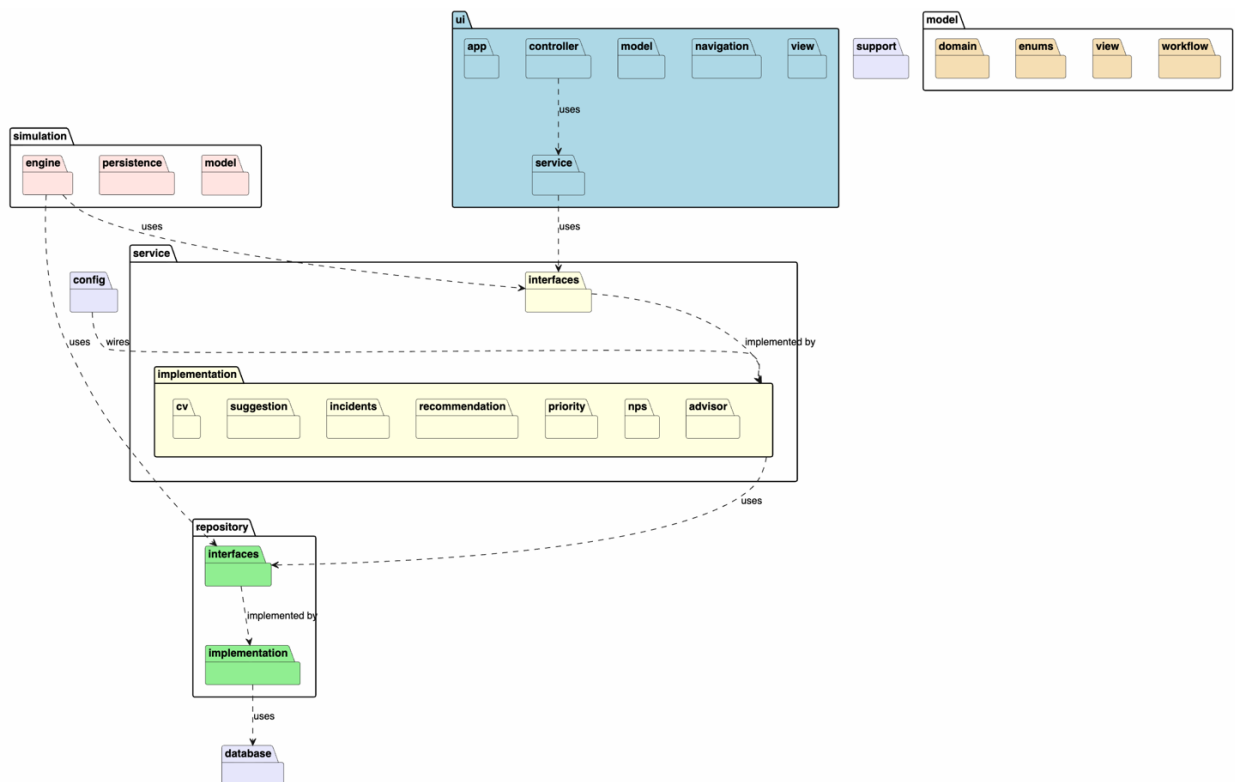


Figure 6: Architecture Structure

2.9 Compliance to Standards and Guidelines

2.9.1 Applied Architecture and Design Guidelines

SOLID Principles:

Principle	Application Examples
Single Responsibility	Each CV scoring rule is its own class. Each suggestion strategy handles one tier. Services are focused on one domain.
Open/Closed	New CV rules can be added without modifying CVScoreCalcServiceImpl (just add to the injected list). New suggestion strategies can be added without changing existing resolvers.
Liskov Substitution	All Incident subclasses (DelayIncident, FeedbackIncident, OnboardingIncident) are substitutable for the base Incident class.
Interface Segregation	Repository interfaces are split into focused contracts: CustomerLookup (read),

	CustomerUpdate (write), CustomerSearch (queries). Clients only depend on what they use.
Dependency Inversion	All services depend on interfaces, not implementations. Dependencies are injected via constructors. The BackendFactory and DashboardApplicationBootstrap handle wiring.

Design Patterns:

Pattern	Where used	Rationale
Strategy	Suggestion system (per package tier)	Different recovery actions for VIP/Gold/Standard customers
Strategy	CV scoring rules	Each rule evaluates independently, new rules can be added
Factory	PackageSuggestionFactory, BackendFactory	Centralized object creation and dependency wiring
Resolver Chain	SuggestionService with list of resolvers	Multiple resolvers attempted in sequence

Additional Standards:

- **DRY:** ResultSetMapper classes (e.g. CustomerResultSetMapper, FlightResultSetMapper) centralise SQL-to-object mapping, eliminating duplication across all repository implementations. Shared query logic is extracted into support classes such as CustomerNoteLoader and FlightViewLoader.
- **KISS:** Complex logic is kept simple wherever possible. Journey status transitions are implemented as a plain EnumMap in JourneyStatusTransitionPolicy rather than a conditional chain. The simulation cleanup service uses a single SQL DELETE with a pattern match, relying on database-level cascade rules instead of multi-step Java logic
- **YAGNI:** Only features required for the prototype were implemented. No real email infrastructure, no user authentication, and no pagination were added, as none of these are needed to demonstrate the core value of the system. Speculative fields such as the advisor workload score were removed once it became clear they served no purpose in the current implementation.
- Java naming conventions: lowercase packages, PascalCase classes, camelCase methods

2.9.2 Deviations

Unit tests are not implemented per instructor guidance. However, the architecture fully supports unit testing through interface-based dependency injection.

AI-based suggestion generation is mocked with a rule-based strategy system rather than actual machine learning models.

3. Design Details

3.1 Business Objects / Database Design

The system uses a SQLite database with the following core tables:

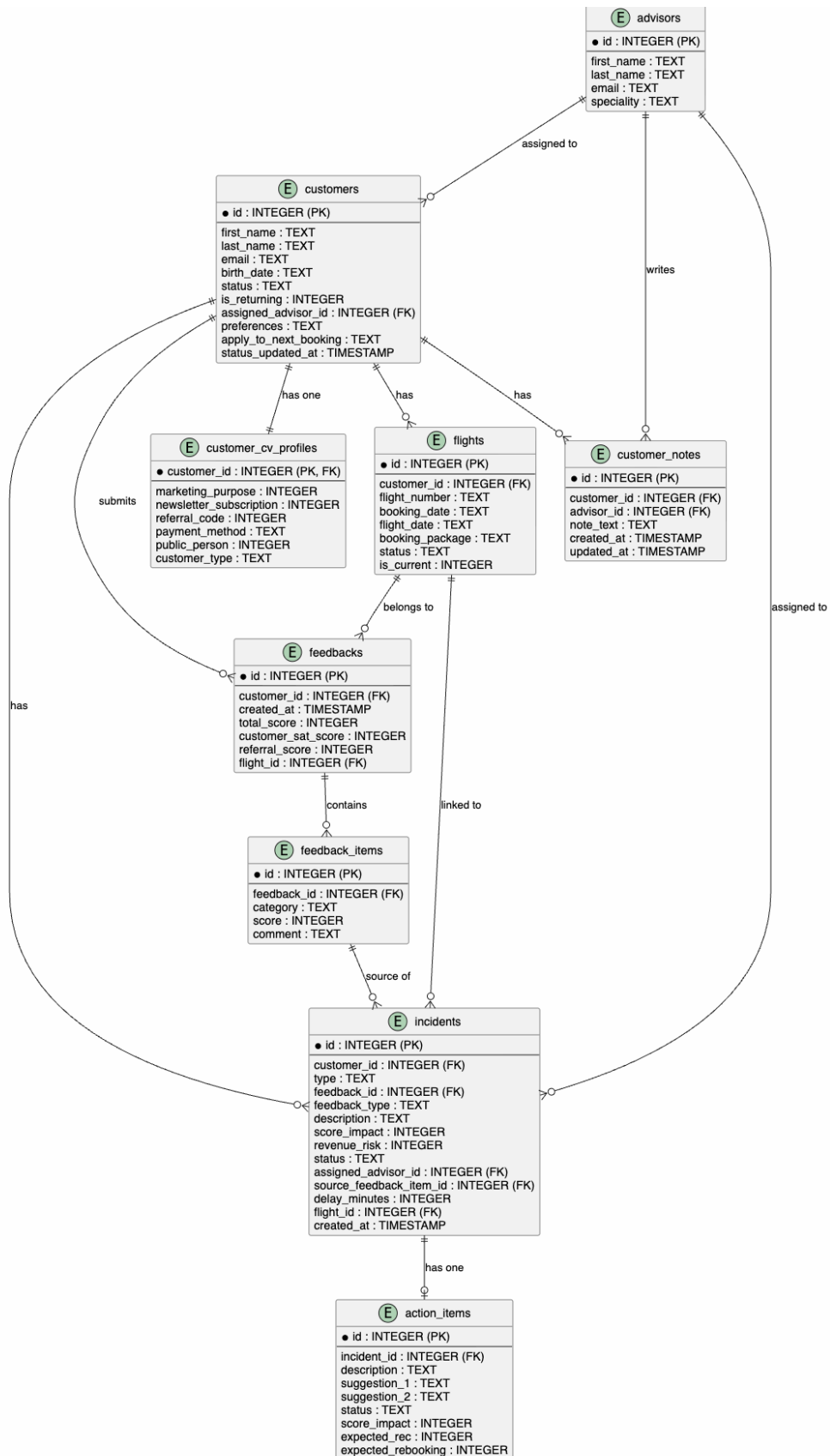


Figure 7: Entity Relationship Diagram of the database schema

Core Entities:

Table	Purpose
customers	Customer personal data, assigned advisor
customer_cv_profiles	Customer value attributes (marketing consent, payment method, etc.)
flights	Flight bookings with package tier, dates
feedbacks	Customer feedback submissions with overall scores
feedback_items	Individual category scores within a feedback (food, hotel, flight)
incidents	Service incidents (delay, feedback, onboarding) with status and impact
action_items	Suggested and completed recovery actions for incidents
advisors	Advisor staff
customer_notes	Free-text notes attached to customers by advisors

Key Relationships:

- A customer has one CV profile, but can have multiple flights and multiple feedbacks (one for each flight)
- An incident belongs to one customer and may have one action item
- A feedback contains multiple feedback items (one per category)

3.2 Testability and Test Environment

The system is designed to be fully testable without requiring a running database. All business services receive their dependencies through constructor injection, which means any interface can be replaced with a mock implementation in tests.

For example, to test the CV score calculation in isolation, one could inject a single mock rule. CV scoring rules are pure functions (given the same CVCustomer input, they always return the same output). Suggestion strategies are stateless and independently testable. The repository interfaces allow complete isolation of database concerns.

3.3 Complex Algorithms and Applied Patterns

The complex algorithms and patterns have been described in detail in section 2.3:

- Section 2.3.1 describes the Suggestion Resolution system (Strategy + Factory + Composite)
- Section 2.3.2 describes the CV Score Calculation (Strategy with injected rule list)

- Section 2.3.3 describes the Priority Scoring algorithm

Moreover the recommendation sending flow is also one of the more complex interactions in the system because it involves two phases. First, the advisor requests a draft email which is pre-populated based on the incident type and customer. Second, after reviewing and optionally editing the content, the advisor sends the recommendation, which atomically stores the sent email as a customer note and closes the incident.

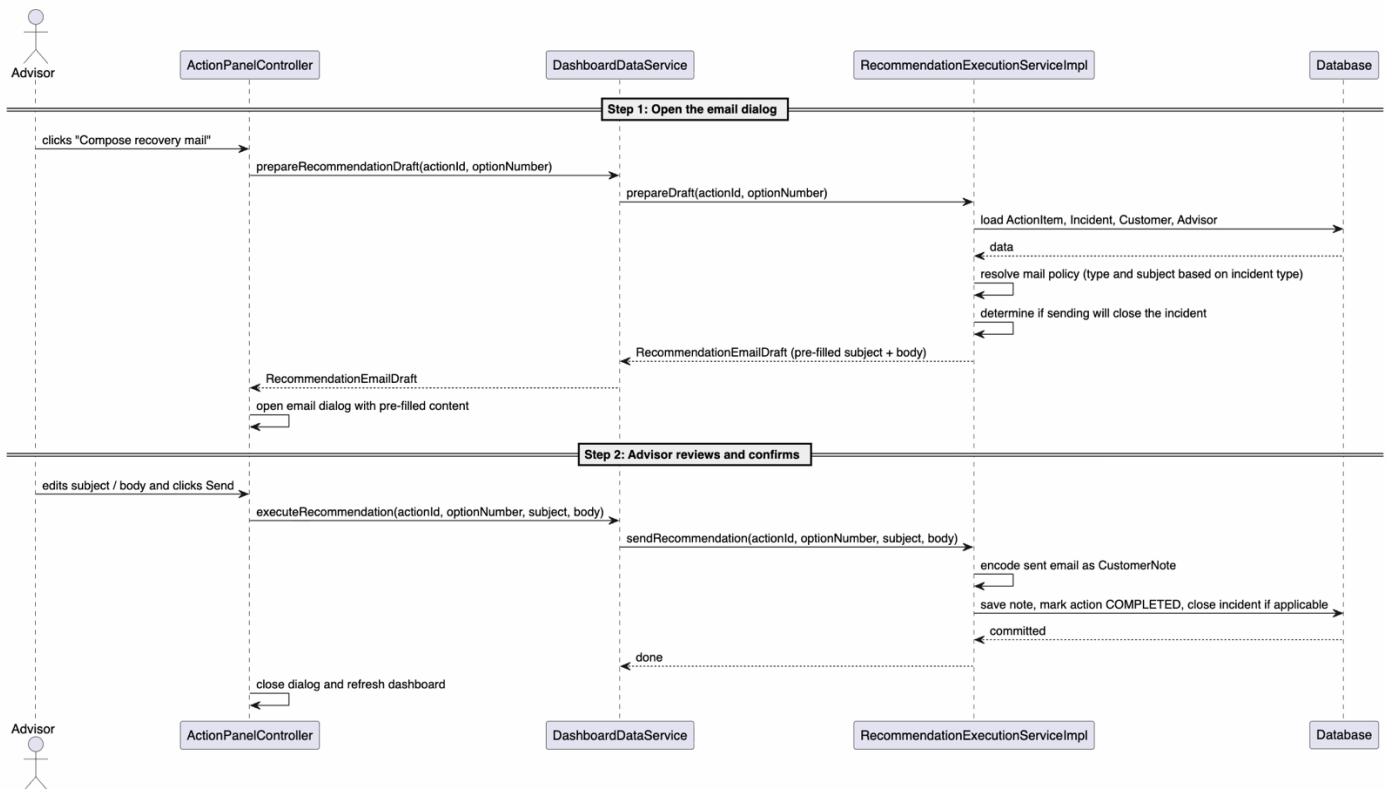


Figure 8: Advisor reviews and sends recommendation

3.4 Design Alternatives and Trade Offs

Decision	Alternative	Rationale
SQLite	H2, PostgreSQL	File-based, no server setup, sufficient for prototype scope
JavaFX	Web UI (React, etc.)	Course requirement; desktop application specified
Rule-based suggestions	ML/AI model	Prototype scope; AI would require training data and infrastructure
1-second refresh timer	WebSocket/push	Desktop app doesn't need network protocols; timer is simpler

3.5 Guide to the implementation

Prerequisites:

- Java 25
- Maven
- IDE (IntelliJ recommended)

Running the application:

IntelliJ will automatically recognize the Maven project and download all dependencies on first import. No additional setup is required, all libraries including the SQLite JDBC driver are declared in pom.xml and will be resolved automatically.

1. Clone the repository
2. Run Main.java
3. The database is automatically created on first startup
4. Select an advisor on the simulation screen (or leave the preselected one)
5. Click “Go to Dashboard” to access the advisor view
6. Optional: Start the simulation to generate test data

Project Structure:

- Source Code: src/main/java
- FXML Views: src/main/resources/ui/view
- Database file: experience.db (auto-generated in project root)

4. Appendix

4.1 Glossary

Term	Definition
CV Score	Customer Value Score: numeric value representing a customer's importance
NPS	Net Promoter Score: percentage of promoters minus percentage of critics
Incident	A service failure event requiring advisor intervention
Action Item	A suggested or completed recovery action for an incident

Package/Tier	Customer booking level: VIP, Gold, or Standard
Journey Status	Customer's current stage in the service lifecycle
Advisor	Staff member responsible for managing customer incidents

4.2 Customizing

The simulation can be configured with: customer creation interval, journey advance interval, delay probability and duration range, feedback score thresholds, and onboarding stuck timeout.

4.3 Supportability Considerations

All database operations are wrapped in try-catch with meaningful error logging via SLF4J. The UI handles exceptions with user-facing error messages rather than exposing stack traces.

4.4 Error Analysis

4.4.1 Debugging

Standard IntelliJ debugger with breakpoints. The service layer is the primary debugging entry point for business logic issues.

4.4.2 Logging and Tracing

SLF4J is used throughout. Repository operations log errors with context (customer ID, operation type). Info-level logs confirm successful data operations

4.4.3 Other Error Analysis Tools

4.5 Other

4.5.1 Use of AI

The overall architecture, design pattern selection, interface design, and business logic were developed by the team without the use of AI. AI coding assistants were used to support implementation in the following specific areas and what is marked in the source code:

Area	What was AI-assisted
Repository layer	ResultSet-to-object mapping in the mapper classes. CustomerViewMapper and

	CustomerResultSetMapper served as the initial AI-generated templates; the remaining mappers were then implemented following the same pattern
Repository support classes	Helper classes for shared query logic (e.g. CustomerNoteLoader, FlightViewLoader) were identified and scaffolded with AI assistance to avoid duplication
Service Layer	Concept of Suggestion and Package Resolver, the recommendation mail draft builder, finding the correct Mail Policy based on an incident (RecommendationMailPolicyResolver) and RecommendationCompletionPolicy which decides if an incident is closed after sending a recommendation mail (not always the case as for some scenarios 2 mails have to get send)
UI Layer	Application wiring in DashboardApplicationBootstrap, FXML controller bindings, dashboard styling and DashboardNodeFactory for dynamic UI elements
Simulation	Randomized customer profile and flight generation in CustomerCreationProcessor, journey advancement logic and feedback generation
Documentation	PlantUML code for sequence diagrams, class diagrams, ER diagram, and package structure diagram. All diagrams were verified against the actual codebase before inclusion
Code Review	Checking compliance with SOLID principles and identifying potential violations